AWS ASSIGNMENT SUBMISSION & AUTOMATED PROCESSING SYSTEM

Expanded Project Documentation & Implementation Report

DOCUMENT DETAIL	CONFIGURATION
Project type	Hands-on AWS cloud project
AWS Region	Asia Pacific (Sydney) — ap-southeast-2
Core services	S3, Lambda, DynamoDB, SNS, EC2, IAM, CloudWatch
Compute environment	Ubuntu 24.04 on EC2 (t3.micro)
Purpose	Academic submission, project demonstration and interview preparation

Scope note: This report preserves the implementation facts and test statuses in the supplied PDF. Additional commands, explanations and design recommendations are labelled as examples or proposed improvements; they are not claims that those features were deployed.

Technical project report

Contents

1. Executive Summary

2. Problem, Objectives and Scope

3. Architecture and Event Flow

4. AWS Services and Design Rationale

5. Resource Inventory

6. Implementation Details

7. Workflow and Data Lifecycle

8. Practical CLI Operations

9. Testing and Evidence

10. Security and Reliability

11. Troubleshooting

12. Limitations and Enhancements

13. Cost Awareness and Cleanup

14. Conclusion and Learning Outcomes

Appendices: Metadata Example and Submission Checklist

1. Executive Summary

The AWS Assignment Submission & Automated Processing System is a cloud-based workflow for collecting student assignment files, recording submission metadata, notifying a reviewer and providing controlled reviewer access. Students upload files to a private Amazon S3 bucket. An S3 ObjectCreated event invokes an AWS Lambda function, which extracts submission details, stores metadata in Amazon DynamoDB and publishes a message to Amazon SNS. An Ubuntu-based Amazon EC2 instance lets the reviewer list and download submitted files through an attached IAM role. Lambda execution logs are available in Amazon CloudWatch Logs.

Capability	How it is provided
File storage	Private S3 bucket with student-specific prefixes.
Automatic processing	S3 ObjectCreated event invokes Lambda for the configured incoming/ prefix.
Metadata tracking	Lambda writes metadata to DynamoDB; the original file remains in S3.
Notification	Lambda publishes to SNS; a confirmed email subscriber receives the message.
Reviewer access	EC2 uses an attached IAM role to list and retrieve S3 objects.
Monitoring	Lambda execution logs are stored in CloudWatch Logs.

Recorded outcomes: EC2 role authentication, S3 listing and test-file download were verified; an SNS email was received; successful Lambda logs were observed. The latest DynamoDB test item and both positive and negative student-prefix access tests remain checks to complete before formal submission.

2. Problem, Objectives and Scope

Problem: Manual assignment collection makes it harder to organise files, identify submissions, notify reviewers and restrict access. This project centralises files in S3 and automates processing after upload.

Objectives

- Provide a private central storage location for assignment files.
- Separate student submissions through individual S3 prefixes and IAM permissions.
- Automatically process new files without maintaining a dedicated processing server.
- Store submission metadata separately from file contents.
- Notify reviewers and provide controlled retrieval access.
- Use CloudWatch Logs to support troubleshooting.

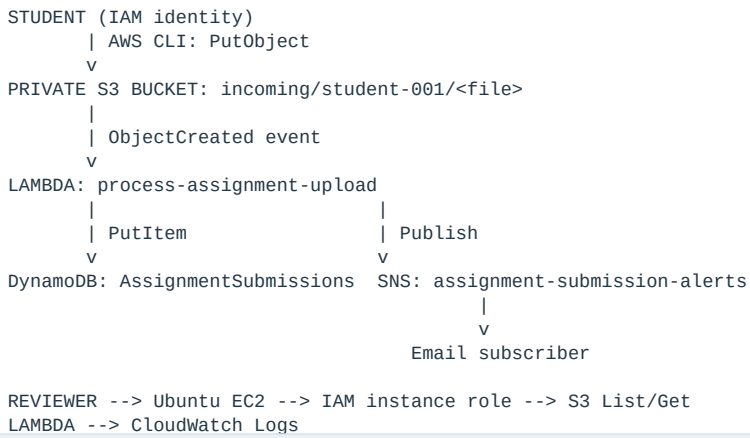
In scope: AWS CLI uploads, S3 event notifications, Lambda processing, DynamoDB metadata, SNS email, EC2 reviewer access, IAM and CloudWatch. **Out of scope:** student website/login portal, automated grading, reviewer dashboard and production-grade resilience.

3. Architecture and Event Flow

S3 is the upload entry point. Lambda reacts to object creation and performs downstream actions. DynamoDB and SNS support metadata and notification tasks. EC2 is a separate reviewer access path; it does not need to process the S3 event itself.

Step	Component	Action
1	Student / AWS CLI	Uploads an assignment using a student IAM identity.
2	Amazon S3	Stores the object under incoming/student-001/.
3	S3 event	Emits an ObjectCreated event for the configured incoming/ prefix.
4	AWS Lambda	Reads the bucket and key, derives details and runs downstream operations.
5a	DynamoDB	Stores a metadata item in AssignmentSubmissions.
5b	SNS	Publishes a notification to the confirmed email subscriber.
6	Reviewer EC2	Uses its instance role to list and download objects.
7	CloudWatch	Stores Lambda execution logs.

Logical architecture



Key distinction: S3 stores the actual file. DynamoDB stores descriptive metadata that helps identify and locate the object. The reviewer downloads the file from S3, not from DynamoDB.

Event sequence in detail

- **Upload:** the student sends an S3 PutObject request to the assigned prefix.
- **Event:** S3 emits an ObjectCreated notification matching the configured prefix filter.
- **Invocation:** Lambda receives event data containing the bucket and object key.
- **Metadata:** the function derives the student folder and filename and creates a submission record.
- **Persistence:** Lambda writes the metadata item to DynamoDB.
- **Notification:** Lambda publishes a message to SNS.
- **Review:** the reviewer uses EC2 role-based credentials to retrieve the object.

4. AWS Services and Design Rationale

Service	Responsibility	Reason for use
Amazon S3	Stores assignment objects.	Object storage with prefixes, encryption, versioning and event notifications.
AWS Lambda	Processes S3 events.	Runs code on demand without a continuously running application server.
DynamoDB	Stores submission metadata.	Managed NoSQL store for structured metadata.
Amazon SNS	Sends email alerts.	Publish/subscribe model separates notifications from processing.
Amazon EC2	Ubuntu reviewer environment.	Provides a Linux host for reviewer CLI operations.
AWS IAM	Controls identities and permissions.	Supports least privilege and role-based temporary credentials.
CloudWatch Logs	Stores Lambda logs.	Helps verify execution and diagnose errors.

S3 concepts used

- **Bucket:** top-level container named aideas-assignment-drop-kk2026.
- **Object:** uploaded file identified by its object key.
- **Prefix:** incoming/student-001/ is a naming convention that behaves like a folder; it is not a separate bucket.
- **Block Public Access:** helps prevent accidental public exposure.
- **Versioning:** can preserve prior object versions when objects are overwritten or deleted.
- **SSE-S3:** server-side encryption using S3-managed keys.

Lambda permission model

Two permissions are involved: a Lambda resource-based permission allows S3 to invoke the function, while the Lambda execution role authorises the function's code to call services such as DynamoDB, SNS and CloudWatch Logs. These controls serve different purposes and should both be checked.

5. Resource Inventory

These names and values are recorded in the supplied project report. Confirm the live AWS console state before redeploying or presenting screenshots.

Resource	Configured value
Region	Asia Pacific (Sydney) — ap-southeast-2
S3 bucket	aideas-assignment-drop-kk2026
Student prefix	incoming/student-001/
Lambda	process-assignment-upload
DynamoDB table	AssignmentSubmissions
Partition key	submissionId (String)
SNS topic	assignment-submission-alerts
Lambda role	role-process-assignment-upload
EC2 reviewer role	role-assignment-reviewer-ec2
Student IAM user	student-001
EC2 instance	assignment-reviewer-server
OS / type	Ubuntu 24.04 / t3.micro
CloudWatch log group	/aws/lambda/process-assignment-upload
Student policy	Student001UploadOnlyPolicy

Configuration dependencies to verify

- S3 event notification targets the correct Lambda function and has the intended incoming/ prefix filter.
- Lambda execution role allows the actual DynamoDB table and SNS topic actions.
- Lambda environment variables contain the correct table name and SNS topic ARN.
- SNS email subscription is Confirmed.
- EC2 has the reviewer role attached and can reach AWS service endpoints.
- All resources and CLI commands use the intended region.

6. Implementation Details

6.1 S3 storage

The bucket is private, Block Public Access is enabled, versioning is enabled and SSE-S3 is configured. Student files are organised under incoming/student-001/. Prefixes do not enforce access by themselves; IAM and bucket policies must enforce the separation.

6.2 Student upload permissions

The original report describes Student001UploadOnlyPolicy as granting s3:PutObject only to objects under the student-specific object ARN. It does not grant read, list, delete or bucket-administration permissions. Review all applicable identity and bucket policies to ensure no broader policy grants extra access.

Illustrative policy shape only — compare against the actual deployed policy before use:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "s3:PutObject",
    "Resource": "arn:aws:s3:::YOUR-BUCKET/incoming/student-001/*"
  }]
}
```

This is an example of a narrow object-level upload permission, not a complete policy audit. Multipart upload or additional features may require further carefully scoped permissions.

6.3 Lambda processing

- Read the bucket name and object key from each S3 event record.
- Derive the student identifier and file name from the key structure.
- Generate a submission identifier and upload timestamp as implemented in the deployed code.
- Write metadata to DynamoDB and publish a notification to SNS.
- Write useful success and failure messages to CloudWatch Logs.

The supplied PDF does not state the Lambda runtime, source code, timeout, memory, retry configuration or exact ID-generation algorithm. Add those details from the live function rather than guessing.

6.4 DynamoDB metadata

AssignmentSubmissions uses submissionId as a String partition key. The described attributes include submission identifier, student identifier, bucket, object key, file name and upload timestamp. File contents are not stored in DynamoDB. Additional indexes would be needed for efficient queries by student or date; the source report does not say these are configured.

6.5 SNS notification

Lambda publishes a message to assignment-submission-alerts. SNS forwards it to the email endpoint after subscription confirmation. A newly added endpoint must confirm its subscription before receiving topic messages.

6.6 EC2 reviewer server

The reviewer host is Ubuntu 24.04 on t3.micro, with role-assignment-reviewer-ec2 attached. The role permits listing the relevant S3 prefix and reading objects under incoming/*. AWS CLI is installed. The source report records successful role authentication, listing and test-file download. No permanent access keys were configured on the server.

6.7 CloudWatch

Lambda logs are available in /aws/lambda/process-assignment-upload. Logs can verify invocations and expose exceptions. Avoid logging credentials, sensitive student details or file contents.

7. Workflow and Data Lifecycle

Actor / service	Input	Output
Student	Local assignment and authorised identity	S3 object in assigned prefix.
S3	New object creation	ObjectCreated event.
Lambda	Bucket, key and event data	Derived submission details and downstream requests.
DynamoDB	Metadata attributes	Item keyed by submissionId.
SNS	Published message	Email to confirmed subscriber.
Reviewer	EC2 session and instance role	List of files and downloaded assignment.
CloudWatch	Execution/application logs	History for diagnosis.

Example lifecycle

A student uploads report.pdf to incoming/student-001/. S3 stores it and sends the configured event. Lambda reads the bucket and key, identifies student-001 and report.pdf, writes metadata to AssignmentSubmissions and publishes a notification. The reviewer later uses the EC2 instance role to retrieve the object. The actual submissionId and timestamp depend on the deployed Lambda code.

At-least-once event delivery

S3 event notifications are designed for at-least-once delivery and may occasionally be duplicated or arrive out of order. Processing should be safe to repeat. A production design can use an idempotency key derived from the object identity and conditional DynamoDB writes to prevent duplicate logical submissions. The original report lists idempotency as a future improvement.

8. Practical CLI Operations

The commands below are examples based on the resource names in the report. Use only an authorised identity and replace local paths as required.

8.1 Configure AWS CLI on the student's computer

```
aws configure
# Set authorised credentials
# Default region: ap-southeast-2
# Output format: json
```

Never include access keys or secrets in screenshots, reports, source code, chat messages or public repositories. Use an approved short-lived credential process where available.

8.2 Upload from Windows PowerShell

```
aws s3 cp "$HOME\Downloads\assignment.pdf" `
  "s3://aideas-assignment-drop-kk2026/incoming/student-001/" `
  --region ap-southeast-2
```

A successful upload confirms the S3 request, not necessarily that all downstream processing completed. Verify Lambda, DynamoDB and SNS separately.

8.3 Verify the active identity

```
aws sts get-caller-identity --region ap-southeast-2
```

On EC2, the returned identity should represent an assumed role associated with the instance profile, not a long-term IAM user's access keys.

8.4 List and download as reviewer

```
aws s3 ls s3://aideas-assignment-drop-kk2026/incoming/ \
  --recursive --region ap-southeast-2

aws s3 cp \
  s3://aideas-assignment-drop-kk2026/incoming/student-001/assignment.pdf \
  ./assignment.pdf --region ap-southeast-2
```

8.5 Inspect Lambda log streams

```
aws logs describe-log-streams \
  --log-group-name /aws/lambda/process-assignment-upload \
  --order-by LastEventTime --descending \
  --region ap-southeast-2
```

8.6 Check DynamoDB items (small test table only)

```
aws dynamodb scan --table-name AssignmentSubmissions --region ap-southeast-2
```

A scan reads across a table and is not an efficient general query pattern at scale. Design keys and indexes around the application's real query needs.

9. Testing and Evidence

Test	Expected result	Recorded status / next action
S3 invokes Lambda	Upload under incoming/ triggers function.	Configured and exercised during upload testing.
SNS email	Confirmed subscriber receives notification.	Email received.
DynamoDB record	Metadata item appears in table.	Confirm latest test item before submission.
CloudWatch logs	Invocation details appear.	Log group and successful logs observed.
EC2 role authentication	STS shows assumed reviewer role.	Verified successfully.
EC2 S3 listing	Reviewer lists incoming/ and student folder.	Verified successfully.
EC2 download	Reviewer downloads a test object.	Verified successfully.
Student prefix restriction	Own-prefix upload succeeds; other prefix is denied.	Policy configured; positive and negative tests pending.

Positive and negative access tests

Positive: use student-001 to upload a harmless test file to incoming/student-001/. Expected result: success.

Negative: using the same identity, attempt to upload a harmless test file to a different student's prefix. Expected result: AccessDenied. Do not use another student's real data.

Also check that student-001 cannot list, read or delete objects if those actions are not intended. Record command, timestamp, identity and result; redact account identifiers and credentials in shared evidence.

Evidence recommended for academic submission

- S3 bucket settings: region, Block Public Access, versioning and encryption.
- S3 object path under incoming/student-001/.
- Lambda trigger configuration and prefix filter.
- Lambda execution role and environment variable names (redact sensitive values).
- DynamoDB table schema and a genuine sample metadata item.
- SNS topic and confirmed subscription status (redact email address if needed).
- EC2 instance details and attached IAM role.
- STS identity, S3 listing and successful download output.
- CloudWatch log entries and student positive/negative access tests.

The original PDF contains no console screenshots. Add genuine screenshots from the current AWS account; do not present illustrative images as deployment evidence.

10. Security and Reliability

Least privilege

- Keep the bucket private and Block Public Access enabled.
- Limit student write access to that student's object prefix.
- Give the reviewer only the list/read permissions needed for review.
- Give Lambda only the DynamoDB, SNS and logging actions required by its code.
- Scope S3's invoke permission to the intended bucket/account where applicable.

Credentials and network access

- Use an EC2 instance role instead of storing long-term AWS keys on the reviewer server.
- Restrict SSH to a trusted source IP; do not expose port 22 to the entire internet.

- Never share secret keys, passwords or private SSH keys.
- Deactivate temporary student keys after testing and rotate exposed credentials.

Data protection and operations

- Use encryption at rest and consider enforcing secure transport.
- Limit access to files, metadata and notification messages.
- Define retention and cleanup rules for files and old object versions.
- Avoid unnecessary personal data in object names and logs.
- Review IAM policies, service quotas and AWS billing periodically.

Reliability

A successful S3 upload does not prove every downstream action succeeded. Lambda may fail during a DynamoDB write or SNS publish. Inspect CloudWatch logs and design retry and recovery behaviour. SQS buffering, a dead-letter queue, idempotent processing and CloudWatch alarms are recommended production improvements.

11. Troubleshooting

Symptom	Likely causes / checks
Upload returns AccessDenied	Check student identity, object key prefix, IAM policy and bucket policy.
Lambda does not run	Check S3 event destination, prefix filter, invoke permission and region.
No DynamoDB item	Check Lambda logs, execution role, table name, key schema and code exception.
No email arrives	Check SNS topic ARN, subscription confirmation, spam folder and publish permission.
EC2 cannot access S3	Check instance profile, role policy, region and network connectivity; run STS identity check.
Can list but cannot download	Check s3:GetObject scope and exact case-sensitive object key.
Duplicate records or emails	Review at-least-once event delivery and add idempotency handling.
No logs visible	Check Lambda logging permissions, log group/stream and selected region.

Troubleshooting sequence

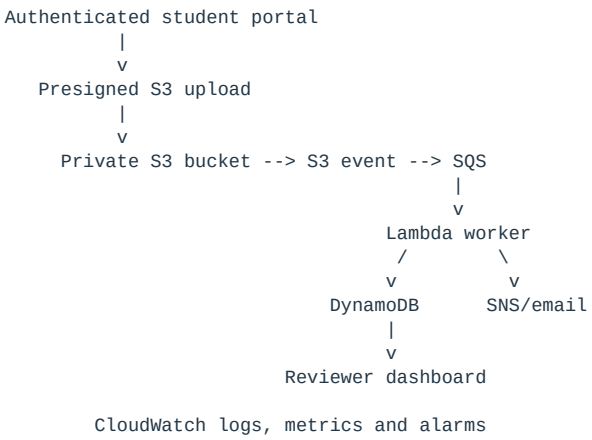
- Identify the failed stage: upload, event, Lambda, DynamoDB, SNS or reviewer retrieval.
- Confirm the AWS account and region before making changes.
- Check service configuration and the relevant IAM permissions.
- Use CloudWatch for Lambda errors and CLI output for identity/access errors.
- Change one item at a time, repeat the test and record the outcome.

12. Limitations and Recommended Enhancements

The current solution is a hands-on demonstration, not a production-hardened platform. It uses direct S3 uploads through AWS CLI and does not include a student portal. The original report identifies duplicate/out-of-order event handling and file validation as future work.

Priority	Enhancement	Benefit
High	Idempotency and conditional DynamoDB writes.	Avoid duplicate logical submissions.
High	Validate file type, size and naming.	Reject invalid or unexpected uploads.
High	CloudWatch alarms for errors and throttling.	Detect failures sooner.
High	SQS buffering and dead-letter queue.	Improve retry and failure handling.
Medium	Presigned URLs or authenticated upload interface.	Enable browser uploads without exposing AWS keys.
Medium	Retention, lifecycle and version cleanup.	Control storage growth.
Medium	DynamoDB indexes for reviewer search.	Support efficient lookup by student/date.
Later	Reviewer dashboard and status tracking.	Simplify review operations.
Later	CloudFormation or Terraform.	Make infrastructure repeatable.
Later	CI/CD and automated tests for Lambda.	Reduce manual deployment errors.

Possible future architecture — proposed, not currently implemented



13. Cost Awareness and Cleanup

AWS managed services can still incur charges. Cost depends on region, storage, request count, Lambda duration, data transfer, log ingestion/retention, EC2 runtime and related resources. This report does not estimate a bill; check the actual account's Billing and Cost Explorer.

- Stop or terminate EC2 when it is no longer required for the demonstration.
- Check attached storage, snapshots and allocated public IP resources.
- Review S3 object versions and configure suitable lifecycle rules.
- Set an appropriate CloudWatch log retention period.
- Avoid unnecessary Lambda invocations and excessive debug logging.

Cleanup caution: Before deleting anything, confirm that submissions and evidence are no longer required. If cleanup is authorised, plan around dependencies and verify the account and region before destructive actions. Do not delete resources still needed for assessment.

14. Conclusion and Learning Outcomes

The project demonstrates an event-driven AWS workflow for assignment collection and reviewer access. S3 provides private file storage, Lambda automates processing, DynamoDB stores metadata, SNS sends notifications, EC2 provides a reviewer environment, IAM enforces access control and CloudWatch supports monitoring. The source report records successful EC2 role-based authentication, S3 listing and test-file download, receipt of the SNS email and observation of successful Lambda logs.

Learning outcomes

- Private object storage and prefix-based organisation in S3.
- Event-driven processing with Lambda.
- Separating binary files from metadata with S3 and DynamoDB.
- Decoupled notifications through SNS.
- Temporary role credentials for EC2 workloads.
- CloudWatch-based operational troubleshooting.
- Importance of least privilege, idempotency, validation, monitoring and cost awareness.